

Project code

```
import pandas as pd
```

```
from sklearn.metrics.pairwise import cosine_similarity
```

```
#
```

```
# STEP 1: Sample user ratings data
```

```
#
```

```
data = {
```

```
    'user': ['A', 'A', 'A', 'B', 'B', 'B', 'C', 'C', 'C'],
```

```
    'movie': ['Inception', 'Titanic', 'Avatar',
```

```
              'Inception', 'Titanic', 'Matrix',
```

```
              'Inception', 'Avatar', 'Matrix'],
```

```
    'rating': [5, 2, 4, 5, 1, 5, 4, 5, 4]
```

```
}
```

```
df = pd.DataFrame(data)
```

```
print("Raw Ratings:")
```

```
print(df)
```

```
print()
```

```
#
```

```
#STEP 2: Build user-movie rating matrix
```

```
#
```

```
table = df.pivot_table(index='user', columns='movie',  
values='rating').fillna(0)
```

```
print("User-Movie Matrix:")
```

```
print(table)
```

```
print()
```

```
#
```

```
#STEP 3: Compute cosine similarity between users
```

```
#
```

```
similarity = cosine_similarity(table)
```

```
sim_df = pd.DataFrame(similarity, index=table.index,  
columns=table.index)
```

```
print("User Similarity Matrix:")
```

```
print(sim_df.round(2))
```

```
print()
```

```
#
```

#STEP 4: Recommend movies for a target user

```
#_____
_____
```

```
def recommend(user, top_n=3):
```

```
    # Get similarity scores for this user (skip self)
```

```
    similar_users = sim_df[user].sort_values(ascending=False)[1:]
```

```
    # Weighted sum of similar users' ratings
```

```
    scores = table.T.dot(similar_users)
```

```
    # Remove movies user already watched
```

```
    already_watched = table.loc[user][table.loc[user] > 0].index
```

```
    scores = scores.drop(already_watched)
```

```
    # Sort and return top N
```

```
    top = scores.sort_values(ascending=False).head(top_n)
```

```
    return top
```

```
#_____
_____
```

#STEP 5: Show recommendations

```
#_____
_____
```

```
user = 'A'
```

```

print(f" Moviesrecommended for {user}:")

recommendations = recommend(user, top_n=3)

for movie, score in recommendations.items():

    print(f" → {movie} (score: {score:.2f})")

print()

user = 'B'

print(f" Moviesrecommended for {user}:")

recommendations = recommend(user, top_n=3)

for movie, score in recommendations.items():

    print(f" → {movie} (score: {score:.2f})")

```

Output of this code :-

Raw Ratings:

	user	movie	rating
0	A	Inception	5
1	A	Titanic	2
...			

User-Movie Matrix:

movie	Avatar	Inception	Matrix	Titanic
user				
A	4.0	5.0	0.0	2.0

B	0.0	5.0	5.0	1.0
C	5.0	4.0	4.0	0.0

User Similarity Matrix:

user	A	B	C
------	---	---	---

user

A	1.00	0.71	0.86
---	------	------	------

B	0.71	1.00	0.69
---	------	------	------

C	0.86	0.69	1.00
---	------	------	------

Movies recommended for A:

→ Matrix (score: 7.13)

Movies recommended for B:

→ Avatar (score: 6.47)